

TECHNISCHE UNIVERSITEIT DELFT

MACHINE LEARNING FOR THE BUILT ENVIRONMENT
(GEO5017)

Assignment 2: Objects Classification from AirBorne LiDAR Data

Authors:

Darius-Eduard Floroiu (5786347)

Belina Aileen Santoso (6369790)

Tejas Ameya Ziegler (6575498)

July 17, 2026



1 Introduction

Modern urban environments demand robust analysis and interpretation of 3D geoinformation. Airborne LiDAR point clouds offer accurate, high-resolution 3D measurements that surpass traditional 2D imagery. A fundamental step in 3D scene understanding is point cloud classification, which aims to assign each point cloud a semantic label.

The goal of this assignment is to focus on classification by predicting the categories of a set of pre-segmented individual urban objects using supervised learning techniques. Specifically, we utilize Support Vector Machines (SVM) and Random Forests (RF), engineer unique geometric features, perform quantitative feature selection, and rigorously evaluate the models through hyperparameter tuning and learning curves. In this report and implementation we exclusively consider the following labels: **building**, **car**, **fence**, **pole** and **tree** and will conduct feature engineering and the analysis of our results correspondingly.

2 Methodology

Feature Engineering

To capture the distinct geometric and topological properties of the urban objects, we engineered 15 initial features. Let a point cloud object be defined as $P = \{p_1, p_2, \dots, p_n\}$ where $p_i = (x_i, y_i, z_i)$. We use these points to get results for our global geometric features. We also extract a local neighborhood of points near the peak of the object using a KD-Tree. For this subset, we compute the covariance matrix and extract its eigenvalues $\lambda_1 \leq \lambda_2 \leq \lambda_3$ and corresponding eigenvectors $\vec{v}_1, \vec{v}_2, \vec{v}_3$. An overview of all features can be seen in [Table 1](#). Note: During implementation, a marginally small constant $\epsilon = 1 \times 10^{-5}$ was added to all denominators to ensure numerical stability and prevent division by zero.

Feature Selection

To select the 4 most effective features, we utilized a filter-based feature selection method based on the Fisher Score (Fisher’s Criterion). For each feature j , we computed the separability ratio J_j using the within-class scatter matrix (S_w) and the between-class scatter matrix (S_b):

$$J_j = \frac{(S_b)_{jj}}{(S_w)_{jj}} \quad (1)$$

Features with higher J scores exhibit high variance between different classes while maintaining low variance within the same class. The 4 features yielding the highest J scores were selected to form the final feature vector used for model training. An overview of all the J scores can be seen in [Table 2](#) (please note that we use bold text for the 4 most effective features).

Experimental Setup

To evaluate the effectiveness of the selected features, we implemented two classifiers: a Support Vector Machine (SVM) and a Random Forest (RF), utilizing the Scikit-Learn library. The dataset was partitioned into a 60% training set and a 40% testing set. A fixed `random_state=42` was maintained across all experiments to guarantee identical data splits for both classifiers, ensuring a fair comparative analysis.

For the SVM classifier, feature standardization was applied to the training set (scaling to a mean of 0 and a variance of 1) to mitigate the algorithm’s sensitivity to feature magnitudes. The RF algorithm, being invariant to feature scaling, utilized the unscaled data.

To determine the optimal hyperparameters for both models, we used a grid search strategy combined with 5-fold cross-validation (`GridSearchCV`). The search spaces defined for the SVM and RF classifiers are detailed in [Table 3](#) and [Table 4](#), respectively. Performance was evaluated using Accuracy and the weighted F1-score to account for potential class imbalances, alongside confusion matrices for detailed error analysis.

3 Experiments and Evaluation

Hyperparameter Tuning and Analysis

Following the grid search procedure outlined in our methodology, we identified the optimal hyperparameters for both classifiers.

Table 1: Combined Global and Local Geometric Features

Name	Type	Formula	Explanation
Height	Global	$H = \max(z_i)$	The maximum elevation of the object.
Root Density	Global	$D_r = \frac{N_{root}}{N}$	Proportion of points within a 2D radius ($r = 0.2$) of the lowest point.
Hull Area	Global	$A = \text{Area}(\text{ConvexHull}_{xy}(P))$	2D area of the object's convex hull on the xy -plane.
Shape Index	Global	$SI = \frac{A}{\text{Perimeter}}$	Ratio of the 2D convex hull's area to its perimeter.
Linearity	Local	$L = \frac{\lambda_3 - \lambda_2}{\lambda_3}$	Measures how much the point distribution resembles a line.
Sphericity	Local	$S = \frac{\lambda_1}{\lambda_3}$	Measures how much the point distribution resembles a sphere.
Planarity	Local	$P = \frac{\lambda_2 - \lambda_1}{\lambda_3}$	Measures how much the point distribution resembles a plane.
Eigenentropy	Local	$E = -\sum_{i=1}^3 e_i \ln(e_i)$	where $e_i = \lambda_i / \sum \lambda$. Quantifies the disorder of the 3D local neighborhood.
Anisotropy	Local	$An = \frac{\lambda_3 - \lambda_1}{\lambda_3}$	Measures the directional variance of the point distribution.
Omnivariance	Local	$O = (\lambda_1 \lambda_2 \lambda_3)^{1/3}$	The geometric mean of the eigenvalues, representing overall volume variance.
Curvature	Local	$C = \frac{\lambda_1}{\lambda_1 + \lambda_2 + \lambda_3}$	Approximates the surface variation or change in the local normal.
Verticality	Local	$V = 1 - \vec{v}_{1,z} $	Deviation of the normal vector from the vertical axis.
Height Diff.	Global	$\Delta H = \max(z_i) - \min(z_i)$	The total vertical span between the highest and lowest points.
Dim. Entropy	Local	$E_{dim} = -\sum \hat{j} \ln(\hat{j})$	where $\hat{j}$ represents normalized vector over linearity, planarity and sphericity. Represents the entropy over normalized L , P , and S .
Local Density	Local	$LPD = \frac{k+1}{\frac{4}{3}\pi d_k^3}$	where k is the number of nearest neighbors and d_k is the distance to the furthest neighbor. Volumetric density around the object's peak.

Table 2: Fisher Score (J) for Engineered Features

Feature Name	Fisher Score (J)
Height	1.1709
Root Density	1.3465
Area	0.1717
Shape Index	0.6382
Linearity	0.9046
Sphericity	0.9438
Planarity	0.6457
Eigenentropy	1.3091
Anisotropy	0.9438
Omnivariance	0.4757
Curvature	1.2939
Verticality	0.6754
Height Diff.	2.0822
Dim. Entropy	0.9278
Local Point Density	0.9237

Support Vector Machine Analysis: The GridSearch identified the optimal configuration as a **linear** kernel with $C = 100$ and $\gamma = \text{scale}$. The success of the linear kernel suggests that our engineered geometric features are already linearly separable in the feature space, making complex kernel transformations unnecessary. The high C value indicates that the model favored a narrower margin to strictly avoid training misclassifications.

Random Forest Analysis: The optimal RF model utilized `n_estimators = 200`, `max_depth = None`, and `min_samples_split = 5`. Increasing the number of trees to 200 improved the ensemble’s stability and generalization. Leaving the maximum depth unconstrained allowed the trees to fully map the feature space, while requiring at least 5 samples to split a node provided just enough regularization to prevent overfitting to noise.

These configurations represent our final recommended models used for all subsequent evaluations.

Table 3: Hyperparameter Search Space for Support Vector Machine (SVM)

Hyperparameter	Tested Values
Kernel (<code>kernel</code>)	<code>rbf</code> , <code>poly</code> , <code>linear</code> , <code>sigmoid</code>
Regularization (<code>C</code>)	0.1, 1, 10, 100
Kernel Coefficient (<code>gamma</code>)	<code>scale</code> , <code>auto</code> , 0.1, 1

Table 4: Hyperparameter Search Space for Random Forest (RF)

Hyperparameter	Tested Values
Number of Trees (<code>n_estimators</code>)	50, 100, 200
Max Depth (<code>max_depth</code>)	<code>None</code> , 10, 20, 30
Min Samples Split (<code>min_samples_split</code>)	2, 5, 10

Learning Curve

As shown in Figure 1, a large accuracy gap between the training and testing sets is observed at lower training sample sizes, indicating initial overfitting for both models. As the training sample size increases, the SVM’s training and testing curves converge, reaching a high test accuracy of approximately 0.96 at 450 samples. Notably, at this maximum split, the SVM test accuracy slightly exceeds the training accuracy, likely a statistical artifact resulting from the small, highly predictable test set size at the 9:1 ratio.

Conversely, while the Random Forest testing accuracy improves steadily to roughly 0.94, its training accuracy remains nearly perfect (1.0) across all sample sizes. This persistent gap indicates that the Random Forest model is still overfitting and memorizing training noise even at maximum data capacity. Overall, the curves suggest that the SVM is the more efficient model in generalizing the geometric features of the urban objects, achieving a better bias-variance trade-off than the Random Forest.

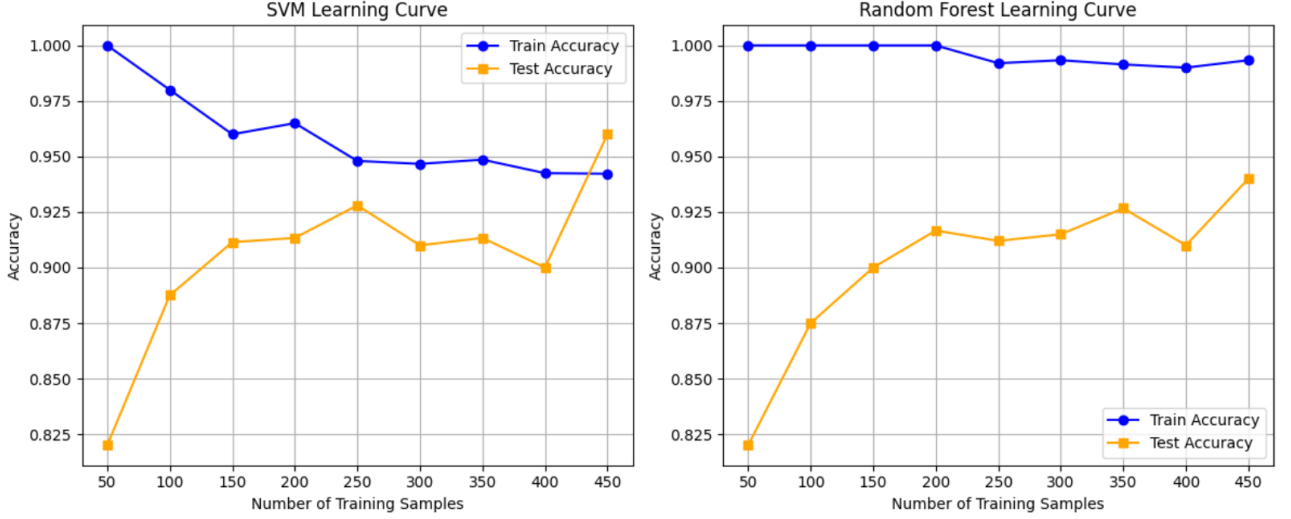


Figure 1: Learning curve of the SVM and Random Forest

Error Analysis and Comparison

To evaluate and compare the models, we selected Accuracy and the weighted F1-score. As mentioned previously, the F1-score is particularly suitable because it balances precision and recall, which helps mitigate potential biases from minor class imbalances in our dataset.

Based on the overall metrics, the classifiers achieved very similar performances. The SVM achieved an accuracy of 0.91, while the RF achieved a slightly higher accuracy of 0.92. Both models achieved a weighted F1-score of 0.91. However, looking closely at their respective confusion matrices (Table 5) reveals some clear differences in how they predict specific categories.

Successes: Both models demonstrate robust performance in predicting **buildings**, **cars**, and **poles**. If we look at the zeros in the matrices, there is a clear separation: cars are never confused with buildings, poles, or trees, and poles are never confused with buildings, cars, or fences. This shows our selected features captured their core shapes well. For example, a pole’s high *verticality* easily separates it from a flat car or a boxy building, while cars likely have a very distinct *Height Diff.* that sets them apart from the taller classes.

Mistakes: The most significant classification challenge for both models is the **fence** category. Fences are frequently misclassified as cars (4 instances for SVM, 5 for RF), and actual buildings are sometimes predicted as fences. Geometrically, this makes sense: short fences can have similar height profiles to cars, and longer ones exhibit strong *planarity* similar to building walls, making them hard to isolate in our feature space. We also see minor, but consistent, mutual confusion between **poles and trees** (likely due to tree trunks sharing similar linearity) and **buildings and trees** (where very dense canopies might mimic a building’s solid *root density*).

Comparison: Even though the overall F1-scores are tied, the SVM with a linear kernel performed slightly better at classifying the most difficult category, fences (26 correct versus 24 for RF). This indicates that our standardized feature space, paired with the SVM’s strict margin ($C = 100$), was slightly better at separating the overlapping boundaries between fences and cars. On the other hand, the RF model showed a slight advantage in identifying trees (40 correct versus 38 for SVM). Ultimately, while RF offers great baseline robustness without needing feature scaling, the SVM performs marginally better in this specific context because it handles our most confused class more effectively.

Table 5: Performance Metrics and Confusion Matrices for SVM and RF Classifiers

Classifier	Accuracy	Weighted F1-score
Support Vector Machine (SVM)	0.91	0.91
Random Forest (RF)	0.92	0.91

SVM Confusion Matrix						RF Confusion Matrix							
Actual	Predicted					Actual	Predicted						
		B	C	F	P		T		B	C	F	P	T
	B	46	0	2	0		2	B	47	0	2	0	1
	C	0	36	2	0		0	C	0	36	2	0	0
	F	1	4	26	0		0	F	2	5	24	0	0
	P	0	0	0	36		3	P	0	0	0	37	2
T	3	0	0	1	38	T	1	0	0	2	39		

Note: B = Building, C = Car, F = Fence, P = Pole, T = Tree. The diagonal elements (bolded) represent correct predictions.

4 Conclusion

In this assignment, we successfully developed a complete pipeline for classifying 3D urban objects from LiDAR point clouds. By engineering a custom set of 15 global and local geometric features and utilizing the Fisher Score to select the four most effective ones, we trained and evaluated both a Support Vector Machine (SVM) and a Random Forest (RF) classifier.

Both models achieved a strong performance with an accuracy and weighted F1-score of 0.91. We conclude that while our selected spatial features are excellent at isolating distinct objects like poles (using verticality) and buildings (using planarity), manual geometric feature engineering struggles to cleanly separate objects with overlapping profiles, such as short fences and cars. Ultimately, the SVM with a linear kernel proved to be the slightly superior model for our specific feature set due to its strict margin optimization on those difficult boundaries.

Suggestions for achieving better results

To get even better results and clear up the remaining confusion between classes, we could try a few different approaches in the future:

1. **Multi-scale neighborhoods:** Instead of just looking at a single local area near the top of the object, we could extract features from different heights (like the base, middle, and top). This would help us capture the full structure of tricky objects like fences.
2. **LiDAR intensity data:** If available, LiDAR intensity data should be incorporated. The laser's return strength provides valuable information for separating natural objects (trees) apart from man-made materials (buildings).
3. **Deep Learning architectures:** Future work could bypass manual feature engineering by utilizing neural networks designed specifically for 3D data, like PointNet. These models automatically figure out the best spatial patterns straight from the raw point clouds.

5 Other information

The code for this experiment can be accessed at <https://github.com/lavaloon2k2/geo5017>. Please note that this is a private repository. If the invitation expired, please contact Darius on Discord (#darius2k2) or by email (dariusfloroiu@gmail.com).

Division of work:

- **Darius:** set-up overleaf and github template, github organization and reviews, report formatting, code implementation (hyperparameter tuning, learning curve), code efficiency, code structure, fix various bugs (for example, eigenvectors not being properly sorted), part of feature selection, report text (introduction, methodology, hyperparameter tuning and analysis, error analysis and comparison, all tables, conclusion)

- **Aileen:** code in feature design and selection, research for literature helpful for feature design. Section 3 learning curve description
- **Tejas:** Debugging code, checking for theoretical errors, structuring report